



Trabalho Prático | DGT2811

Bárbara Gabriela Madruga Mecabô

2024.10.16936-5

CAMPUS BIGUAÇU - SC

Desenvolvimento Full Stack

Desenvolvimento Backend Corporativo com Java e Cloud – 9001 – 2025.4

Objetivo da Prática

A presente prática teve como objetivo implementar um sistema cadastral web utilizando a plataforma Jakarta EE, integrando as tecnologias JPA (Java Persistence API), EJB (Enterprise Java Beans), Servlets e JSP, com persistência de dados em banco SQL Server e execução no servidor GlassFish 6.2.1.

Conforme descrito no material da disciplina, o trabalho prático visou desenvolver as camadas de persistência, negócio e controle de um sistema corporativo, estruturando o projeto em módulos (EAR, EJB e WAR) no NetBeans e aplicando os conceitos de arquitetura multicamadas e padrão MVC. Nesse contexto, foram realizadas as seguintes etapas:

- Configuração do pool de conexões JDBC no GlassFish e integração com o banco de dados SQL Server na Azure;
- Geração das entidades JPA a partir da base de dados;
- Implementação da camada de negócio por meio de Session Beans (EJB);
- Implementação da camada de controle utilizando Servlets, incluindo o padrão Front Controller;
- Desenvolvimento das páginas JSP para apresentação dos dados;
- Aplicação do framework Bootstrap para melhoria do layout e responsividade;

- Deploy e execução da aplicação no servidor, validando a integração entre banco de dados, camada de negócio e interface web.

Além dos aspectos técnicos, a prática buscou reforçar a organização estrutural do projeto e o uso adequado do versionamento com Git, promovendo controle, rastreabilidade e padronização no desenvolvimento.

Ao final, a aplicação foi executada com sucesso, demonstrando a correta comunicação entre as camadas do sistema e o funcionamento integrado das tecnologias propostas no roteiro da disciplina.

Link do repositório no Git: <https://github.com/barbaragmm/cadastroee-jakarta>

1º Procedimento | Camadas de Persistência e Controle

AbstractFacade.java:

```
package cadastroee.controller;

import jakarta.persistence.EntityManager;
import java.util.List;

public abstract class AbstractFacade<T> {

    private Class<T> entityClass;

    public AbstractFacade(Class<T> entityClass) {
        this.entityClass = entityClass;
    }

    protected abstract EntityManager getEntityManager();

    public void create(T entity) {
        getEntityManager().persist(entity);
    }
}
```

```

    }

    public void edit(T entity) {
        getEntityManager().merge(entity);
    }

    public void remove(T entity) {
        getEntityManager().remove(getEntityManager().merge(entity));
    }

    public T find(Object id) {
        return getEntityManager().find(entityClass, id);
    }

    public List<T> findAll() {
        jakarta.persistence.criteria.CriteriaQuery cq =
getEntityManager().getCriteriaBuilder().createQuery();

        cq.select(cq.from(entityClass));

        return getEntityManager().createQuery(cq).getResultList();
    }

    public List<T> findRange(int[] range) {
        jakarta.persistence.criteria.CriteriaQuery cq =
getEntityManager().getCriteriaBuilder().createQuery();

        cq.select(cq.from(entityClass));

        jakarta.persistence.Query q = getEntityManager().createQuery(cq);

        q.setMaxResults(range[1] - range[0] + 1);

        q.setFirstResult(range[0]);

        return q.getResultList();
    }

    public int count() {

```

```

        jakarta.persistence.criteria.CriteriaQuery cq =
getEntityManager().getCriteriaBuilder().createQuery();

        jakarta.persistence.criteria.Root<T> rt = cq.from(entityClass);

        cq.select(getEntityManager().getCriteriaBuilder().count(rt));

        jakarta.persistence.Query q = getEntityManager().createQuery(cq);

        return ((Long) q.getSingleResult()).intValue();
    }

```

ProdutoFacade.java:

```

package cadastroee.controller;

import cadastroee.model.Produto;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

@Stateless
public class ProdutoFacade extends AbstractFacade<Produto> implements
ProdutoFacadeLocal {

    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;

    @Override
    protected EntityManager getEntityManager() {
        return em;
    }

    public ProdutoFacade() {
        super(Produto.class);
    }
}

```

ProdutoFacadeLocal.java:

```

package cadastroee.controller;

import cadastroee.model.Produto;
import jakarta.ejb.Local;
import java.util.List;

@Local
public interface ProdutoFacadeLocal {

    void create(Produto produto);

    void edit(Produto produto);

    void remove(Produto produto);

    Produto find(Object id);

    List<Produto> findAll();
}

```

```

        List<Produto> findRange(int[] range);

        int count();
    }

```

Produto.java:

```

package cadastroee.model;

import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import jakarta.xml.bind.annotation.XmlRootElement;
import java.io.Serializable;

@Entity
@Table(name = "Produto")
@XmlRootElement
@NamedQueries({
    @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM Produto p"),
    @NamedQuery(name = "Produto.findByCodigo", query = "SELECT p FROM Produto p WHERE p.codigo = :codigo"),
    @NamedQuery(name = "Produto.findByNome", query = "SELECT p FROM Produto p WHERE p.nome = :nome"),
    @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p FROM Produto p WHERE p.quantidade = :quantidade")})
public class Produto implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @Basic(optional = false)
    @NotNull
    @Column(name = "codigo")
    private Integer codigo;
    @Size(max = 40)
    @Column(name = "nome")
    private String nome;
    @Column(name = "quantidade")
    private Integer quantidade;
    @Column(name = "precoVenda")
    private Float precoVenda;

    public Produto() {
    }

    public Produto(Integer codigo) {
        this.codigo = codigo;
    }

    public Integer getCodigo() {
        return codigo;
    }

    public void setCodigo(Integer codigo) {
        this.codigo = codigo;
    }

    public String getNome() {

```

```

        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public Integer getQuantidade() {
        return quantidade;
    }

    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }

    public Float getPrecoVenda() {
        return precoVenda;
    }

    public void setPrecoVenda(Float precoVenda) {
        this.precoVenda = precoVenda;
    }

    @Override
    public int hashCode() {
        int hash = 0;
        hash += (codigo != null ? codigo.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        if (!(object instanceof Produto)) {
            return false;
        }
        Produto other = (Produto) object;
        if ((this.codigo == null && other.codigo != null) || (this.codigo !=
null && !this.codigo.equals(other.codigo))) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "cadastroee.model.Produto[ codigo=" + codigo + " ]";
    }
}

```

ServletProduto.java:

```

package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import jakarta.ejb.EJB;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class ServletProduto extends HttpServlet {

    @EJB

```

```

        ProdutoFacadeLocal facade;

        @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {

            response.setContentType("text/html;charset=UTF-8");

            try (PrintWriter out = response.getWriter()) {

                out.println("<html>");
                out.println("<head><title>Lista de Produtos</title></head>");
                out.println("<body>");
                out.println("<h1>Lista de Produtos</h1>");
                out.println("<ul>");

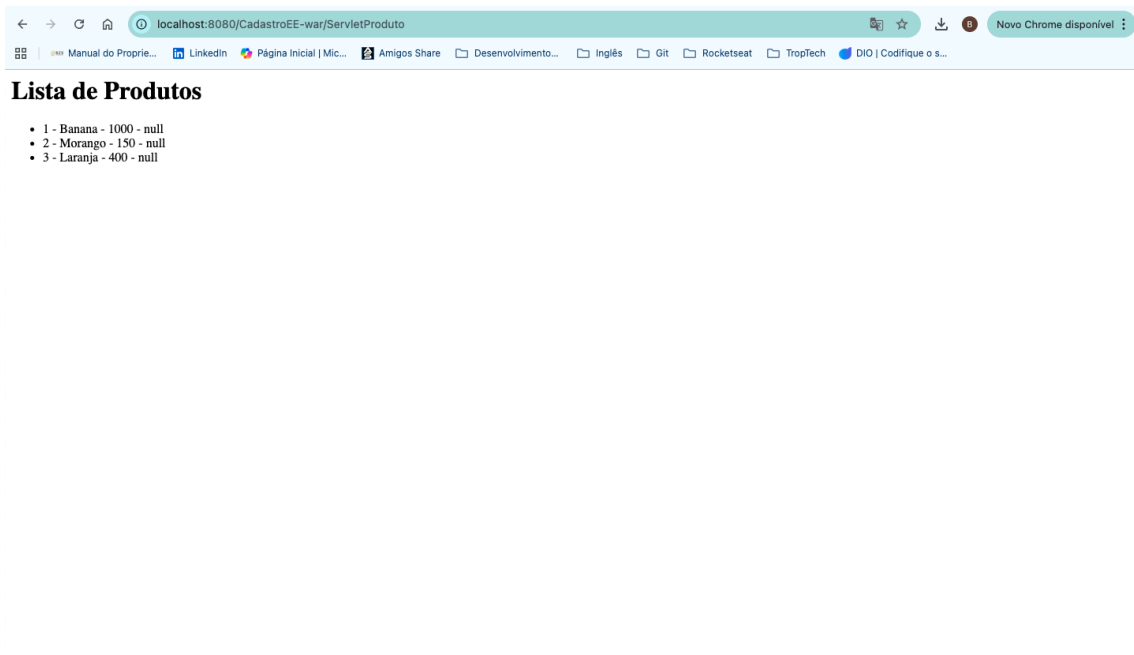
                var lista = facade.findAll();

                for (var p : lista) {
                    out.println("<li>"
                        + p.getCodigo() + " - "
                        + p.getNome() + " - "
                        + p.getQuantidade() + " - "
                        + p.getPrecoVenda()
                        + "</li>");
                }

                out.println("</ul>");
                out.println("</body>");
                out.println("</html>");
            }
        }
    }
}

```

Resultado da execução do código:



Análise e Conclusão

- a. Como é organizado um projeto corporativo no NetBeans?

Um projeto corporativo no NetBeans é organizado de forma modular, separando responsabilidades em diferentes camadas e módulos, o que facilita a manutenção, reutilização de código e escalabilidade da aplicação.

No contexto desta prática, foi utilizado o modelo **Enterprise Application**, que gera automaticamente três módulos principais: Projeto EAR (Enterprise Archive) que é o módulo principal que empacota e integra todos os demais módulos da aplicação; Módulo EJB (CadastroEE-ejb) sendo responsável pela camada de negócio e persistência; e Módulo Web (CadastroEE-war) que é responsável pela camada de apresentação.

- b. Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java?

A **JPA** é responsável pela camada de persistência, seu papel é realizar o mapeamento objeto-relacional (ORM), permitindo que as tabelas do banco de dados sejam representadas como classes Java (entidades). Já o **EJB** atua na camada de regras de negócio. Os Session Beans encapsulam a lógica de acesso aos dados e centralizam as operações CRUD.

- c. Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB?

O NetBeans contribui significativamente para a melhoria da produtividade no desenvolvimento de aplicações corporativas com JPA e EJB ao oferecer ferramentas automatizadas, assistentes de configuração e integração direta com servidores de aplicação.

Durante a prática, foi possível observar que o ambiente fornece recursos que reduzem a complexidade técnica e evitam a escrita manual de grande parte da configuração necessária, como: geração automática de entidades JPA a partir do banco de dados, geração automática de Session Beans (EJB) com métodos padrão CRUD, assistente para criação de Servlets, gerenciamento visual de conexões com banco de dados, organização automática em módulos (EAR, EJB e WAR).

- d. O que são Servlets, e como o NetBeans oferece suporte à construção desse tipo de componentes em um projeto Web?

Servlets são componentes Java executados no servidor que têm como principal função receber, processar e responder a requisições HTTP em aplicações Web. Eles atuam como intermediários entre o cliente e a camada de negócio da aplicação, sendo responsáveis por controlar o fluxo das requisições e gerar respostas dinâmicas.

O NetBeans oferece suporte significativo à construção de Servlets por meio de assistentes automáticos e integração com o servidor de aplicação. Ao criar um novo Servlet, a IDE: gera automaticamente a estrutura básica da classe; inclui os métodos *doGet()* e *doPost()*; adiciona a configuração no arquivo *web.xml*, quando necessário; organiza o arquivo dentro do módulo Web (WAR) e oferece suporte a *imports* automáticos e validação em tempo real.

Além disso, o NetBeans permite executar e testar o Servlet diretamente no navegador por meio da integração com o GlassFish, facilitando o ciclo de desenvolvimento e teste.

- e. Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs?

A comunicação entre Servlets e Session Beans ocorre por meio da injeção de dependência gerenciada pelo container do servidor de aplicação. No ambiente Jakarta EE, essa integração é realizada utilizando a anotação *@EJB*, que permite ao Servlet acessar um EJB sem necessidade de instanciá-lo manualmente.

O pool de EJBs funciona como um conjunto de instâncias prontas para uso, permitindo que múltiplas requisições sejam atendidas de forma eficiente e segura. Quando o Servlet recebe uma requisição HTTP, ele chama métodos do EJB, e o EJB, por sua vez, utiliza a JPA para acessar o banco de dados.

Link do repositório no Git: <https://github.com/barbaragmm/cadastroe-jakarta>

2º Procedimento | Interface Cadastral com Servlet e JSPs

ServletProdutoFC.java:

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.RequestDispatcher;
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.util.List;

public class ServletProdutoFC extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String acao = request.getParameter("acao");
        String destino = "ProdutoLista.jsp";

        if (acao == null || acao.equals("listar")) {

            List<Produto> lista = facade.findAll();
            request.setAttribute("lista", lista);
            destino = "ProdutoLista.jsp";
        } else if (acao.equals("formIncluir")) {
            destino = "ProdutoDados.jsp";
        } else if (acao.equals("formAlterar")) {

            Integer id = Integer.parseInt(request.getParameter("id"));
            Produto p = facade.find(id);
            request.setAttribute("produto", p);
            destino = "ProdutoDados.jsp";
        } else if (acao.equals("excluir")) {

            Integer id = Integer.parseInt(request.getParameter("id"));
            Produto p = facade.find(id);
            facade.remove(p);

            List<Produto> lista = facade.findAll();
            request.setAttribute("lista", lista);
            destino = "ProdutoLista.jsp";
        } else if (acao.equals("alterar")) {

            Integer id = Integer.parseInt(request.getParameter("id"));
            Produto p = facade.find(id);

            p.setNome(request.getParameter("nome"));

            p.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));
            p.setPrecoVenda(Float.parseFloat(request.getParameter("precoVenda")));

            facade.edit(p);

            List<Produto> lista = facade.findAll();
            request.setAttribute("lista", lista);
            destino = "ProdutoLista.jsp";
        } else if (acao.equals("incluir")) {
```

```

        Produto p = new Produto();

        p.setCodigo(Integer.parseInt(request.getParameter("id")));
        p.setNome(request.getParameter("nome"));

        p.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));
        p.setPrecoVenda(Float.parseFloat(request.getParameter("precoVenda")));

        facade.create(p);

        List<Produto> lista = facade.findAll();
        request.setAttribute("lista", lista);
        destino = "ProdutoLista.jsp";
    }

    RequestDispatcher rd = request.getRequestDispatcher(destino);
    rd.forward(request, response);
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    processRequest(request, response);
}
}

```

ProdutoLista.jsp:

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="cadastror.model.Produto"%>

<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Lista de Produtos</title>
    </head>
    <body>

        <h1>Lista de Produtos</h1>

        <!-- Link para inclusão -->
        <a href="ServletProdutoFC?acao=formIncluir">
            Incluir Novo Produto
        </a>

        <br><br>

        <table border="1">
            <tr>
                <th>Código</th>
                <th>Nome</th>
                <th>Quantidade</th>
                <th>Preço</th>
            </tr>

```

```

        <th>Ações</th>
    </tr>

    <%
        List<Produto> lista = (List<Produto>)
request.getAttribute("lista");

        if (lista != null) {
            for (Produto p : lista) {
    %>
    <tr>
        <td><%= p.getCodigo()%></td>
        <td><%= p.getNome()%></td>
        <td><%= p.getQuantidade()%></td>
        <td><%= p.getPrecoVenda()%></td>
        <td>
            <a href="ServletProdutoFC?acao=formAlterar&id=<%=
p.getCodigo()%>">
                Alterar
            </a>
            |
            <a href="ServletProdutoFC?acao=excluir&id=<%=
p.getCodigo()%>">
                Excluir
            </a>
        </td>
    </tr>
    <%
        }
    %>

    </table>

</body>
</html>

```

ProdutoDados.jsp:

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastror.model.Produto"%>

<%
    Produto produto = (Produto) request.getAttribute("produto");
    boolean alterar = (produto != null);
    String acao = alterar ? "alterar" : "incluir";
    %>

<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Dados do Produto</title>

        <style>
            body {
                font-family: Arial, sans-serif;
            }

            h1 {
                margin-bottom: 20px;
            }

            .form-container {
                display: flex;

```

```

        align-items: center;
        gap: 15px;
    }

    label {
        font-weight: bold;
    }

    input[type="text"],
    input[type="number"] {
        padding: 5px;
        width: 150px;
    }

    input[type="submit"] {
        padding: 6px 12px;
        cursor: pointer;
    }
</style>
</head>

<body>

    <h1>Dados do Produto</h1>

    <form action="ServletProdutoFC" method="post">

        <input type="hidden" name="acao" value="<%= acao%>"/>

        <% if (alterar) {%>
        <input type="hidden" name="id" value="<%=
produto.getCodigo()%>"/>
        <% }%>

        <div class="form-container">

            <div>
                <label>Nome:</label><br>
                <input type="text" name="nome"
                value="<%= alterar ? produto.getNome() : ""%>"
required/>
            </div>

            <div>
                <label>Quantidade:</label><br>
                <input type="number" name="quantidade"
                value="<%= alterar ? produto.getQuantidade() :
""%>" required/>
            </div>

            <div>
                <label>Preço de Venda:</label><br>
                <input type="number" step="0.01" name="precoVenda"
                value="<%= alterar ? produto.getPrecoVenda() :
""%>" required/>
            </div>

            <% if (!alterar) { %>
            <div>
                <label>Código:</label><br>
                <input type="number" name="id" required/>
            </div>
            <% }%>

            <div style="margin-top: 18px;">
                <input type="submit"
                value="<%= alterar ? "Alterar Produto" :
"Adicionar Produto"%>"/>
            </div>

```

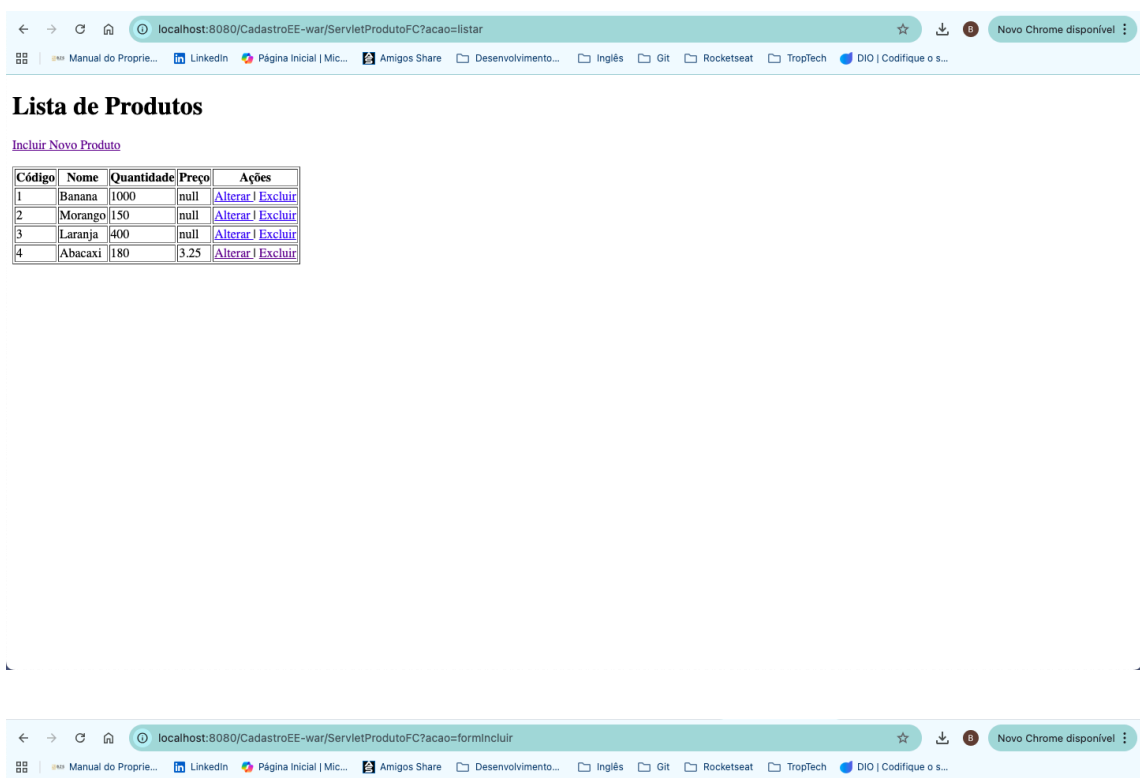
```
        </div>

    </form>

    <br>
    <a href="ServletProdutoFC?acao=listar">Voltar para Lista</a>

</body>
</html>
```

Resultado da execução do projeto:



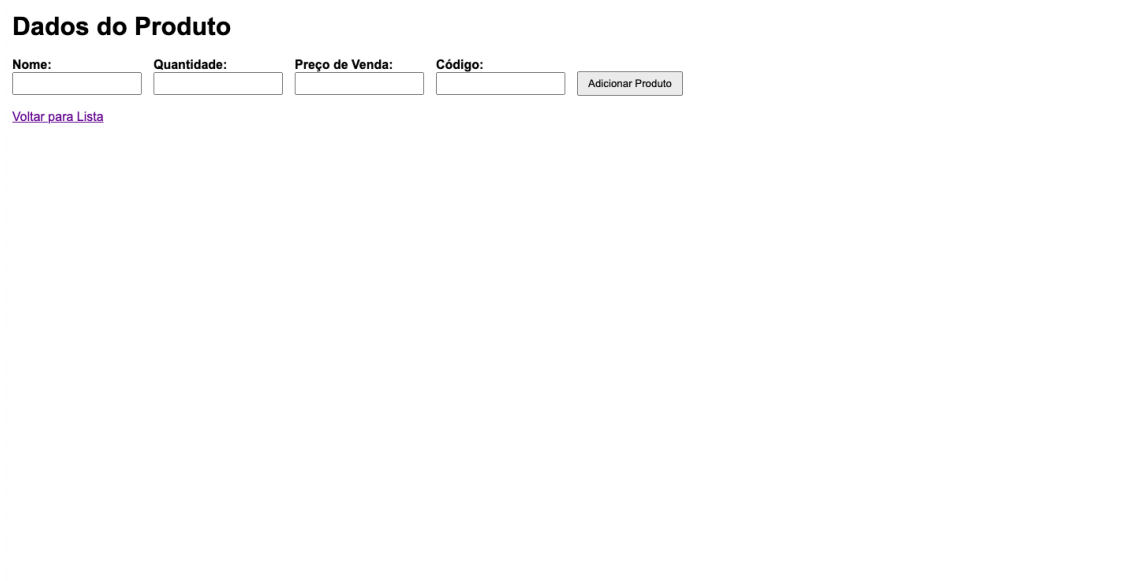
localhost:8080/CadastroEE-war/ServletProdutoFC?acao=listar

Manual do Proprie... LinkedIn Página Inicial | Mic... Amigos Share Desenvolvimento... Inglês Git Rocketseat TropTech DIO | Codifique o s...

Lista de Produtos

[Incluir Novo Produto](#)

Código	Nome	Quantidade	Preço	Ações
1	Banana	1000	null	Alterar Excluir
2	Morango	150	null	Alterar Excluir
3	Laranja	400	null	Alterar Excluir
4	Abacaxi	180	3.25	Alterar Excluir



localhost:8080/CadastroEE-war/ServletProdutoFC?acao=formIncluir

Manual do Proprie... LinkedIn Página Inicial | Mic... Amigos Share Desenvolvimento... Inglês Git Rocketseat TropTech DIO | Codifique o s...

Dados do Produto

Nome: Quantidade: Preço de Venda: Código:

[Voltar para Lista](#)

Análise e Conclusão

- a. Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?

O padrão Front Controller é um padrão de projeto utilizado em aplicações Web no qual todas as requisições do usuário são centralizadas em um único componente controlador. Esse controlador é responsável por receber a requisição, interpretar seus parâmetros, decidir qual lógica de negócio deve ser executada e encaminhar a resposta para a camada de apresentação adequada.

Na arquitetura MVC, o Front Controller atua como o Controller central, promovendo a separação clara entre:

- **Model:** representa as regras de negócio e o acesso aos dados;
- **View:** responsável pela interface com o usuário;
- **Controller:** responsável por controlar o fluxo da aplicação.

Na prática desenvolvida, o padrão foi implementado por meio do Servlet *ServletProdutoFC*, que passou a ser o único ponto de entrada para as operações relacionadas à entidade Produto. Esse Servlet recebe um parâmetro chamado “acao”, cujo valor determina qual funcionalidade será executada.

- b. Quais as diferenças e semelhanças entre Servlets e JSPs?

Servlets e JSPs são tecnologias da plataforma Java utilizadas para o desenvolvimento de aplicações Web dinâmicas. Ambas são executadas no servidor e fazem parte da especificação Jakarta Servlet, sendo processadas por um container Web, como o GlassFish.

Semelhanças:

- Ambas executam no lado do servidor.
- São capazes de receber requisições HTTP e gerar respostas dinâmicas.
- Podem acessar objetos como *request*, *response*, *session* e *application*.
- Permitem integração com a camada de negócio (EJB, JPA etc.).
- Fazem parte da arquitetura MVC em aplicações Java Web.

Diferenças:

- Servlets são classes Java e são utilizados principalmente para controle de fluxo e processamento de requisições. Além disso, contém lógica de controle e integração com a camada de negócio. E também são mais adequados para atuar como Controller na arquitetura MVC.
 - JSP (JavaServer Pages) são páginas com sintaxe semelhante a HTML que permitem a inserção de código Java dentro da página. São mais indicadas para apresentação de dados ao usuário, visto que facilitam a construção da interface visual. E atuam como View na arquitetura MVC.
- c. Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

No desenvolvimento Web com Servlets, existem duas formas principais de encaminhar uma requisição para outro recurso: **redirecionamento simples** e **encaminhamento interno (forward)**.

- **Redirecionamento simples – *response.sendRedirect()***
 - Envia uma resposta ao navegador solicitando que ele faça uma nova requisição HTTP;
 - O navegador altera a URL na barra de endereço;
 - Os dados da requisição original são perdidos;
 - Há duas requisições:
 - Do cliente para o servidor.
 - Nova requisição do cliente após o redirecionamento.
- **Encaminhamento interno – *RequestDispatcher.forward()***
 - O redirecionamento ocorre internamente no servidor;
 - Não há nova requisição do navegador;
 - A URL permanece a mesma;
 - Os atributos adicionados ao objeto *request* permanecem disponíveis;
 - Existe apenas uma requisição.

E quanto aos parâmetros e atributos no *HttpServletRequest*:

- Parâmetros (*getParameter()*): são enviados pelo cliente; podem vir da URL (GET) ou de formulários (POST); representam dados de entrada; são sempre do tipo *String*.
- Atributos (*setAttribute()* / *getAttribute()*): são definidos no lado do servidor; servem para compartilhar dados entre o Servlet e a JSP; existem apenas durante o ciclo da requisição; podem armazenar qualquer tipo de objeto.

Link do repositório no Git: <https://github.com/barbaragmm/cadastroee-jakarta>

3º Procedimento | Melhorando o Design da Interface

ServletProdutoFC.java:

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.RequestDispatcher;
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.util.List;

public class ServletProdutoFC extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String acao = request.getParameter("acao");
        String destino = "ProdutoLista.jsp";

        if (acao == null || acao.equals("listar")) {

            List<Produto> lista = facade.findAll();
            request.setAttribute("lista", lista);
            destino = "ProdutoLista.jsp";
        } else if (acao.equals("formIncluir")) {

            destino = "ProdutoDados.jsp";
        } else if (acao.equals("formAlterar")) {

            String idStr = request.getParameter("id");

            if (idStr != null && !idStr.isEmpty()) {
                Integer id = Integer.parseInt(idStr);
                Produto p = facade.find(id);
            }
        }
    }
}
```

```

        request.setAttribute("produto", p);
    }

    destino = "ProdutoDados.jsp";
} else if (acao.equals("excluir")) {

    String idStr = request.getParameter("id");

    if (idStr != null && !idStr.isEmpty()) {
        Integer id = Integer.parseInt(idStr);
        Produto p = facade.find(id);

        if (p != null) {
            facade.remove(p);
        }
    }

    List<Produto> lista = facade.findAll();
    request.setAttribute("lista", lista);
    destino = "ProdutoLista.jsp";
} else if (acao.equals("alterar")) {

    String idStr = request.getParameter("id");

    if (idStr != null && !idStr.isEmpty()) {

        Integer id = Integer.parseInt(idStr);
        Produto p = facade.find(id);

        if (p != null) {

            String nome = request.getParameter("nome");
            String quantidadeStr =
request.getParameter("quantidade");
            String precoStr = request.getParameter("precoVenda");

            p.setNome(nome);

            if (quantidadeStr != null && !quantidadeStr.isEmpty()) {
                p.setQuantidade(Integer.parseInt(quantidadeStr));
            }

            if (precoStr != null && !precoStr.isEmpty()) {
                p.setPrecoVenda(Float.parseFloat(precoStr));
            }

            facade.edit(p);
        }
    }

    List<Produto> lista = facade.findAll();
    request.setAttribute("lista", lista);
    destino = "ProdutoLista.jsp";
} else if (acao.equals("incluir")) {

    Produto p = new Produto();

    String idStr = request.getParameter("id");
    String nome = request.getParameter("nome");
    String quantidadeStr = request.getParameter("quantidade");
    String precoStr = request.getParameter("precoVenda");

    if (idStr != null && !idStr.isEmpty()) {
        p.setCodigo(Integer.parseInt(idStr));
    }

    p.setNome(nome);

    if (quantidadeStr != null && !quantidadeStr.isEmpty()) {

```

```

        p.setQuantidade(Integer.parseInt(quantidadeStr));
    }

    if (precoStr != null && !precoStr.isEmpty()) {
        p.setPrecoVenda(Float.parseFloat(precoStr));
    }

    facade.create(p);

    List<Produto> lista = facade.findAll();
    request.setAttribute("lista", lista);
    destino = "ProdutoLista.jsp";
}

RequestDispatcher rd = request.getRequestDispatcher(destino);
rd.forward(request, response);
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    processRequest(request, response);
}
}

```

ProdutoLista.jsp:

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="cadastroroe.model.Produto"%>

<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Listagem de Produtos</title>

        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.cs
s"
        rel="stylesheet"

integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBu
jZqJLB"
        crossorigin="anonymous">

        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.m
in.js"

integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPVl7YYwJrWVcXK/BmnVDxM+D2sc
QbITxI"
        crossorigin="anonymous"></script>
    </head>

    <body class="container">

```

```

<h1 class="mt-4">Listagem de Produtos</h1>

<a href="ServletProdutoFC?acao=formIncluir"
  class="btn btn-primary m-2">
  Novo Produto
</a>

<table class="table table-striped">

  <thead class="table-dark">
    <tr>
      <th>#</th>
      <th>Nome</th>
      <th>Quantidade</th>
      <th>Preço de Venda</th>
      <th>Opções</th>
    </tr>
  </thead>

  <tbody>

    <%
      List<Produto> lista = (List<Produto>)
request.getAttribute("lista");

      if (lista != null) {
        for (Produto p : lista) {
          <%>

          <tr>
            <td><%= p.getCodigo() %></td>
            <td><%= p.getNome() %></td>
            <td><%= p.getQuantidade() %></td>
            <td><%= p.getPrecoVenda() %></td>
            <td>
              <a href="ServletProdutoFC?acao=formAlterar&id=<%=
p.getCodigo() %>"
                class="btn btn-primary btn-sm">
                  Alterar
              </a>

              <a href="ServletProdutoFC?acao=excluir&id=<%=
p.getCodigo() %>"
                class="btn btn-danger btn-sm">
                  Excluir
              </a>
            </td>
          </tr>

          <%
            }
          <%>

        </tbody>
      </table>

    </body>
  </html>

```

ProdutoDados.jsp:

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastroee.model.Produto"%>

```

```

<%
    Produto produto = (Produto) request.getAttribute("produto");
    boolean alterar = (produto != null);
    String acao = alterar ? "alterar" : "incluir";
%>

<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Dados do Produto</title>

        <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.cs
s"
            rel="stylesheet"

integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBu
jZqJLB"
            crossorigin="anonymous">

        <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.m
in.js"

integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2sc
QbITxI"
            crossorigin="anonymous"></script>
        </head>

    <body class="container mt-4">

        <h1>Dados do Produto</h1>

        <form action="ServletProdutoFC" method="post">

            <input type="hidden" name="acao" value="<%= acao%>"/>

            <% if (alterar) {%>
            <input type="hidden" name="id" value="<%=
produto.getCodigo()%>"/>
            <% }%>

            <div class="mb-3">
                <label class="form-label">Nome</label>
                <input type="text"
                    name="nome"
                    class="form-control"
                    value="<%= alterar ? produto.getNome() : ""%>"
                    required>
            </div>

            <div class="mb-3">
                <label class="form-label">Quantidade</label>
                <input type="number"
                    name="quantidade"
                    class="form-control"
                    value="<%= alterar ? produto.getQuantidade() : ""%>"
                    required>
            </div>

            <div class="mb-3">
                <label class="form-label">Preço de Venda</label>
                <input type="number"
                    step="0.01"
                    name="precoVenda"
                    class="form-control"
                    value="<%= alterar ? produto.getPrecoVenda() : ""%>"
                    required>

```

```

</div>

<% if (!alterar) { %>
<div class="mb-3">
  <label class="form-label">Código</label>
  <input type="number"
    name="id"
    class="form-control"
    required>
</div>
<% }%>

<button type="submit" class="btn btn-primary">
  <%= alterar ? "Alterar Produto" : "Adicionar Produto"%>
</button>

<a href="ServletProdutoFC?acao=listar" class="btn btn-secondary
ms-2">
  Voltar
</a>

</form>

</body>
</html>

```

Resultado da execução do projeto:

The screenshot shows a web application running on a local server. The page title is 'Listagem de Produtos'. At the top left, there is a blue button labeled 'Novo Produto'. Below this is a table with the following columns: '#', 'Nome', 'Quantidade', 'Preço de Venda', and 'Opções'. The table contains 5 rows of data:

#	Nome	Quantidade	Preço de Venda	Opções
1	Banana	1000	5.0	Alterar Excluir
2	Morango	150	6.7	Alterar Excluir
3	Laranja	400	1.8	Alterar Excluir
4	Abacaxi	180	3.2	Alterar Excluir
5	Manga	180	2.3	Alterar Excluir

The browser's address bar shows the URL 'localhost:8080/CadastroEE-war/ServletProdutoFC'. The browser interface includes standard navigation buttons and a notification for 'Novo Chrome disponível'.

localhost:8080/CadastroEE-war/ServletProdutoFC?acao=formIncluir

Dados do Produto

Nome

Quantidade

Preço de Venda

Código

[Adicionar Produto](#) [Voltar](#)

localhost:8080/CadastroEE-war/ServletProdutoFC?acao=formAlterar&id=5

Dados do Produto

Nome

Quantidade

Preço de Venda

[Alterar Produto](#) [Voltar](#)

Análise e Conclusão

- Como o framework Bootstrap é utilizado?

Sua utilização ocorre por meio da inclusão de seus arquivos CSS e JavaScript no projeto, podendo ser feita via CDN (Content Delivery Network) ou download local dos arquivos. Nesta prática, a integração foi realizada via CDN, inserindo os links oficiais do Bootstrap dentro da seção `<head>` das páginas JSP.

Após a inclusão, o framework é utilizado através da aplicação de classes CSS pré-definidas diretamente nos elementos HTML. Essas classes permitem aplicar estilos prontos a botões, melhorar a apresentação de tabelas, padronizar formulários e até organizar o layout com containers e espaçamentos.

b. Por que o Bootstrap garante a independência estrutural do HTML?

O Bootstrap garante a independência estrutural do HTML porque sua utilização baseia-se na aplicação de classes CSS previamente definidas, sem exigir alterações na estrutura lógica do documento. Ou seja, o desenvolvedor mantém a organização semântica do HTML (como `<div>`, `<table>`, `<form>`, `<input>`, `<button>`) e apenas adiciona classes que controlam a apresentação visual.

Essa abordagem promove a separação entre estrutura (HTML), estilo (CSS do Bootstrap) e comportamento (JavaScript).

c. Qual a relação entre o Bootstrap e a responsividade da página?

A responsividade de uma página Web refere-se à capacidade da interface se adaptar automaticamente a diferentes tamanhos de tela, como desktops, tablets e smartphones, mantendo usabilidade e organização visual.

O Bootstrap contribui diretamente para a responsividade por meio de um sistema de grid flexível e de classes utilitárias baseadas em breakpoints. O framework foi desenvolvido com a abordagem **mobile-first**, ou seja, os estilos são pensados inicialmente para dispositivos móveis e depois adaptados para telas maiores.

Link do repositório no Git: <https://github.com/barbaragmm/cadastroe-jakarta>

Conclusão

O Trabalho Prático possibilitou a construção de uma aplicação Web corporativa completa utilizando Jakarta EE, integrando JPA, EJB, Servlets, JSP e Bootstrap. A atividade permitiu aplicar, de forma prática, conceitos de arquitetura em camadas e o padrão MVC, além do uso do padrão Front Controller para centralização do controle das requisições.

O desenvolvimento evidenciou a importância da organização do projeto, da consistência entre as camadas da aplicação e da correta configuração do ambiente (servidor e banco de dados). Problemas enfrentados durante a implementação, como ajustes de parâmetros e integração com o banco (utilizar banco de dados SQL Server na Azure por conta de incompatibilidade de sistema operacional), contribuíram para consolidar o entendimento sobre o funcionamento real de uma aplicação corporativa.

A utilização do Bootstrap demonstrou como é possível melhorar a interface e garantir responsividade sem alterar a estrutura lógica da aplicação, reforçando a separação entre backend e apresentação.

De forma geral, a prática cumpriu seu objetivo ao proporcionar uma experiência completa de desenvolvimento, integrando múltiplas tecnologias e aproximando o ambiente acadêmico de um cenário profissional.